

level slider control on computer screen. Now it is ready for connecting the sensor such as temperature sensor or LDR to sensor input. The sensor output voltage level will be shown in the PC screen as shown in Fig. 4.

Software

The software for interfacing sensor with ADC of AVR microcontroller are written in 'C' using IAR Embedded Workbench integrated development environment. IAR Embedded Workbench is being developed by IAR Systems and ATMELE developers in parallel and hence it generates the optimised code which uses full 'C' coding capabilities of AVR devices. AVR development tools for embedded systems can be downloaded for free from IAR website www.iar.com. For details of IAR Embedded Workbench, you may refer to 'A Beginners' Guide to ATMELE AVR Development' article published in January issue of EFY.

ADC channel in AVR is initialised by setting ADC enable (ADEN) bit in ADC control and status register A

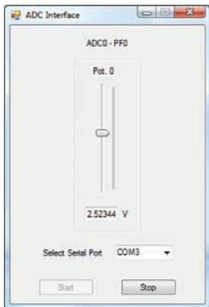


Fig. 4: ADC interface windows

(ADCSRA), which can also be used to set the frequency for ADC conversion. ADC multiplexer selection register (ADMUX) is used to select the channel to be used for ADC input. Setting the ADC start conversion (ADSC) bit starts conversion and sets the ADC interrupt flag (ADIF) in ADCSRA register on completion. The converted data can be read from ADC register.

Source code

The source code is demonstrated using one of the ADC inputs of ATmega2560 microcontroller ADC channel. Following the steps demonstrated in the article, you can create your C# application for interfacing the circuit board. You should, however, have a version of Microsoft Visual Studio .NET. You can use C# for effective serial communication in your own development board or the current ADC application for receiving serial data.

To create the application run Microsoft Visual Studio and create a new C# based Windows form application. One is expected to get a blank windows form1. Using the toolbox, drag and drop the controls to create a front-end tool like the one shown in Fig. 4. Set the properties of form1 as MaximizeBox = False, StartPosition

as CenterScreen, FormBorderStyle as FixedDialog and name it as 'ADC Interface'. Similarly, for the 'trackBarPot1' set the maximum value as 255, orientation as 'Vertical' and 'TickFrequency' as 5. Add texts to the buttons as 'Start' and 'Stop'. This completes the front-end development. Now event-driven codes are to be inserted for the controls. To do this, double click on the controls to enter the coding environment for each event. The code should be as listed below:

```
using System;
using System.Windows.Forms;

namespace Simple_Serial
{
    public partial class Form1 : Form
    {
        int RxData;

        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

RxData is the variable to store the data received through serial port, which is declared in the form as given below:

```
private void Form1_Load(object sender, EventArgs e)
{
    PortCombo.Items.AddRange(System.
    IO.Ports.SerialPort.GetPortNames());
    if (PortCombo.Items.Count == 0)
        MessageBox.Show("No
    Serial port is available!");
}
```

The above mentioned subroutine executes as soon as the form is loaded. It obtains all the available port names from the system and adds it to the drop-down combo list from where one can select the port where the device is connected. On selecting the port name from the combo box the following code executes. It initialises the 115.2kbps baud rate. The start button is enabled and stop button is disabled.

```
private void PortCombo_
```

PARTS LIST

Semiconductors:

IC1	- ATmega2560 microcontroller
IC2	- MAX232 RS-232 driver
IC3	- 7805, 5V regulator
DI-D4	- 1N4007 rectifier diode
LED1, LED2	- 5mm LED

Resistors (all 1/4-watt, $\pm 5\%$ carbon):

R1	- 330-ohm
R2	- 10-kilo-ohm
R3	- 470-ohm
VR1, VR2	- 10-kilo-ohm preset

Capacitors:

C1	- 1000 μ F, 25V electrolytic
C2	- 0.1 μ F ceramic disk
C3, C4	- 22pF ceramic disk
C5	- 10 μ F, 16V electrolytic
C6-C10	- 1 μ F, 16V electrolytic

Miscellaneous:

X1	- 230V AC primary to 9V, 500mA secondary transformer
S1	- Push-to-on tactile switch
CON1	- 2-pin berg strip male connector
CON2	- 6-pin berg strip male connector
CON3	- 9-pin D-type connector
JP1-JP2	- 3-pin berg strip male connector with shorting jumper